IMPROVING TEXT-TO-MUSIC GENERATION MODEL TRAINING THROUGH PROMPT-CONSISTENCY CONDITIONING

Anonymous Authors
Anonymous Affiliations
anonymous@ismir.net

ABSTRACT

Automatic music captioning models are widely used to generate captions for the training of text-to-music (TTM) generation models when the annotated text-audio pairs are not readily available, yet these auto-generated captions could be inconsistent due to the fact that one audio may have multiple reasonable captions, posing challenges for training effectiveness. To address this issue, we propose prompt-consistency conditioning, which provides the TTM model with an additional condition that shows the caption consistency of the auto-generated captions. By introducing this condition, during training time, the TTM model has access to information on the consistency of the input text prompt, which provides a better guidance on music generation. Experiment results confirm that the proposed prompt-consistency conditioning method improves the Meta Audiobox Aesthetics production quality scores and the CLAP text-audio semantic similarity score over the baseline without the proposed prompt-consistency conditioning. A paired preference study further supports the improvement, with listeners showing a clear preference for the proposed method in both audio quality (+0.467 CMOS) and prompt-following ability (+0.289 CMOS). These results indicate that conditioning the TTM model on the prompt-consistency signal helps alleviate the limitations of inconsistent automatic music captions during unsupervised TTM model training.

1. INTRODUCTION

Text-to-music (TTM) generation has advanced rapidly in recent years, driven by powerful generative architectures such as autoregressive language models [1–5], flow matching models [6–8], and diffusion models [9–13]. Training these models requires large amounts of audio-caption pairs [1, 2], where the captions are used as input text prompts to the TTM model for training. However, publicly available high-quality audio-caption pairs are scarce [14], and manually annotating music at scale is prohibitively expensive. This poses a severe challenge to the training of TTM models, especially for academic researchers who do not have large-scale internal datasets. To address this issue, two methods have been adopted. First, one can automatically create music captions from music metadata using a Large Language Model (LLM) [12, 14]. Second, one can directly employ an automatic music captioning model to automatically generate captions [15, 16], further eliminating the requirement of metadata, allowing a fully *unsupervised* learning pipeline for TTM training.

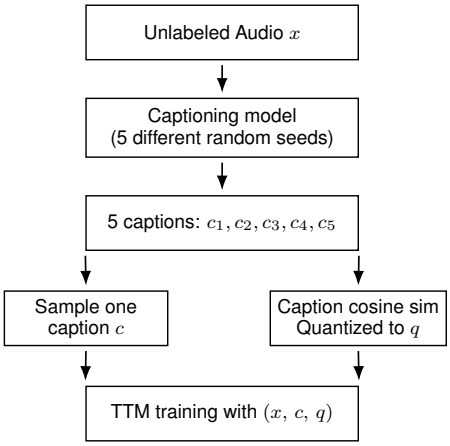

Figure 1. An overview of the proposed PromptCC method for unsupervised TTM training. For each audio clip, we generate five captions using a music captioning model with different random seeds. We compute the prompt-consistency score q as the quantized mean pairwise semantic cosine similarity of these captions, and pair a randomly selected caption with q as the input to the TTM model. At inference, q is fixed to 9 (highest consistency).

Although the use of a music captioning model allows for unsupervised TTM training, it also introduces concerns about the quality and characteristics of the captions. An auto-generated music caption may not fully capture the details of the music. Moreover, a music clip may have multiple captions that are all reasonable. For example, "A masterclass in explosive dynamics, shifting seamlessly from delicate whispers to thunderous symphonic roars" and "The rubato breathes life into every phrase, pulling and pushing the tempo with captivating, organic intensity" both describe a professional performance of Johannes Brahms' *Hungarian Dance No.5* well. A music captioning model may be equally likely to choose between the two captions, which brings up the consistency issue of the captions. A TTM model trained on auto-generated captions thus has to be robust enough to mitigate the biases and inconsistencies

© . Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** , "Improving Text-to-music Generation Model Training Through Prompt-consistency Conditioning", submitted to *ISMIR*, 2026.

propagated by the music captioning model.

In this regard, we propose *prompt-consistency conditioning* (hereinafter abbreviated as *PromptCC*), an approach that provides the consistency score of different captions generated from the same target audio as an additional condition to the TTM model, thereby allowing the TTM model to understand how consistent an auto-generated prompt is. Specifically, for each audio clip, we apply the same music captioning model five times to create five different captions. Then, we compute their mean pairwise text similarity, which we term as *prompt-consistency*, and quantize this score into one of 10 levels, which is injected as a learnable embedding into a TTM model via adaptive layer normalization (AdaLN). This signal informs the TTM model about the degree of agreement among generated captions of the same audio, thereby mitigating the consistency issue of the auto-generated prompts.

In the experiments, we use the LP-MusicCaps model [17] to generate captions for the MTG-Jamendo dataset [18], which are then used to train MeanAudio [8], a lightweight text-to-audio generation model, for TTM. Objective evaluations indicate that the proposed PromptCC method improves TTM performance in the MusicCaps [3] dataset (with manually-written text prompts) using Meta Audiobox Aesthetics [19], CLAP [20], and PE-AV [21] as objective evaluators. Subjective evaluations further show that the proposed PromptCC method improves the Comparison Mean Opinion Score (CMOS) in audio quality and prompt-following ability by $+0.467$ and $+0.289$ over a baseline without using PromptCC, respectively. These results confirm that adding the prompt-consistency condition to the TTM model effectively mitigates the consistency issue of auto-generated captions, improving the training of a TTM model in the unsupervised learning scenario.

2. RELATED WORK

2.1 Text-to-Music Generation

Text-to-music (TTM) generation aims at generating a music signal from a text prompt. Recent TTM systems can be broadly categorized into autoregressive [1–4] and non-autoregressive methods [6–13]. Autoregressive methods such as MusicGen [1] employ an autoregressive language model on discrete audio tokens, allowing controllable music generation from text. On the other hand, non-autoregressive methods typically perform music generation by denoising a randomly-sampled audio prior, either through flow matching [6–8] or diffusion mechanism [9–13, 22, 23]. These methods achieve higher generation speed due to their non-autoregressive property, which enables parallel generation. For example, Stable Audio Open [6] applies latent diffusion in a compressed audio space, while MusicFlow [7] introduces a cascaded flow matching framework that separately models semantic and acoustic features. In this work, we adopt MeanAudio [8], a lightweight flow matching model, in the experiments.

2.2 Auto-Generated Captions for Text-to-Audio Generation Training

Since manually-written captions are scarce, many previous TTM or text-to-audio generation systems rely heavily on automatically generated text captions for model training. Noise2Music [14] pairs a music-language model with a LLM to synthesize pseudo-captions for diffusion training, while MeanAudio [8] adopts datasets with text prompts that are automatically rewritten from audio tags [17, 24]. CosyAudio [25] further proposes training a music captioning model to refine these pseudo-captions. During this automatic prompt generation process, if there are audio tags available, the generated text prompts would be more reliable. But when the audio tags are not available, i.e., a fully unsupervised learning scenario, the generated text prompts may exhibit bias or consistency issues [19, 26]. In this work, we address this issue through the proposed PromptCC method.

2.3 Quality-Aware Conditioning for TTM Training

Several recent works have explored using quality or preference signals as additional conditions to text-to-music (TTM) generation models. QA-MDT [26] conditions a masked diffusion transformer on pseudo-MOS quality scores by injecting discretized quality scores as prefix tokens. MR-FlowDPO [27] introduces *reward prompting*, in which three continuous reward values, namely text-audio alignment (CLAP score [20]), audio production quality (Audiobox Aesthetics score [19]), and audio semantic consistency (HuBERT likelihood [28]), are prepended to the text prompt. The model thus learns to generate music conditional on explicit target reward levels. Inspired by these works, we also propose injecting an additional condition (prompt-consistency condition) into the TTM training loop. However, the proposed method does not capture the *quality* of audio or text captions, but rather captures the *consistency* of text captions, which helps mitigate the consistency issue of a music captioning model.

3. METHOD

An overview of the proposed method is shown in Figure 1. This work considers an *unsupervised learning* scenario in which a large-scale dataset with audio and text caption pairs is not available. In this case, one has to apply a music captioning model to generate text prompts to form a training dataset. To this end, we propose adding prompt-consistency conditioning to the TTM model to mitigate the consistency issue of the auto-generated text prompts. Since the proposed method does not require any text annotations, it has the potential to be scaled up to the TTM training on much larger unlabeled datasets.

3.1 Backbone Model

We adopt MeanAudio as the backbone model [8], a lightweight flow matching-based [29] transformer model designed for text-to-audio generation. During training, the

audio waveforms are first converted to Mel spectrograms and encoded into a latent sequence $\mathbf{x} \in \mathbb{R}^{N \times d}$ via a pre-trained variational autoencoder, where N is the sequence length and d is the latent dimension. Then, given a text prompt c, the model first employs two pretrained text encoders, namely FLAN-T5 [30] and CLAP [20], to extract text features $\mathbf{y}_c$ from c. The model is trained to generate $\mathbf{x}$ from $\mathbf{y}_c$ and a randomly sampled prior ϵ through the conditional flow matching mechanism. In particular, adaptive layer normalization (AdaLN) is applied to inject text features $\mathbf{y}$ into the generation process. We choose MeanAudio as our TTM backbone model for its high training and inference efficiency (achieving a real-time factor of 0.013 [8]), along with its modest model size (120M), which allows for training and evaluation on a consumer GPU.

3.2 Prompt-Consistency Condition

When only a dataset with unlabeled audio is available, an alternative method for TTM training is to employ an automatic music captioning model such as LP-MusicCaps [17] to generate captions, which are paired with the audio to form the training dataset. However, these auto-generated captions may not be consistent due to 1) the ambiguity of the music captioning task and 2) the potential errors made by the captioning model. Our preliminary experiment using LP-MusicCaps further reveals the effect of the second issue. For example, using different random seeds for captioning, an audio in the MTG-Jamendo dataset [18] is captioned as "...synth pad chords, laser synth bass and punchy kick" or "...it could be used in the soundtrack of a horror movie or a video game". Upon manual inspection, we found that both captions are correct, as the audio features synth, bass, and punchy kick components, as well as a pervasive horror-themed tone. Such an ambiguity may negatively affect the subsequent TTM training, which heavily relies on these auto-generated captions.

To address this issue, we propose repeating the inference of the music captioning model for $K = 5$ times with different random seeds to generate five captions for each audio clip. Then, given these captions $\{c_1, \ldots, c_K\}$, we compute the *prompt-consistency score* as the mean pairwise cosine similarity of their sentence embeddings:

$$s = \frac{2}{K(K-1)} \sum_{i<j} \cos\_\mathrm{sim}\big(\phi(c_i),\, \phi(c_j)\big), \qquad (1)$$

where $\phi(\cdot)$ denotes a sentence encoder, which computes a fixed-dimension embedding for each text caption, while $\cos\_\mathrm{sim}$ denotes the cosine similarity function. In practice, we use Sentence-BERT [31] as the sentence encoder. [1] The resultant s value thus quantifies the degree of prompt consistency across all prompts generated by the music captioning model.

After obtaining the s score, a natural approach is to remove clips with less consistent auto-generated captions, i.e., s is lower than a specific threshold, in order to mitigate the issue of prompt consistency. However, this hard-removal approach could remove a large fraction of the training data, potentially damaging the diversity of audio and text prompts. Our ablation study (as will be shown in Section 4) also shows that this hard-filtering approach actually results in worse TTM performance. Therefore, we instead retain all training samples and treat the s value as another conditional signal, which is then fed into the TTM model for generation. The idea behind this *prompt-consistency conditioning* approach is that, by providing the TTM model with the consistency information, it can implicitly handle the prompt consistency issue. Consequently, the TTM model can still learn well from audio with less consistent text prompts, achieving better performance compared to a baseline model without the use of this conditioning mechanism.

3.3 Prompt-Consistency Embedding

To feed the prompt-consistency condition into the TTM model, inspired by QA-MDT [26], we further propose quantizing continuous s values into 10 equally spaced consistency levels (uniform quantization). [2] For clarity, we refer to the quantized s values as the q-values, where $q \in \{0, 1, \ldots, 9\}$. An additional index $q = 10$ is reserved as a null token for the classifier-free guidance (CFG) mechanism [32] during MeanAudio's training [8].

Then, to feed the q information into the TTM model, we introduce a learnable embedding layer **pc_embed** $\in \mathbb{R}^{D \times 11}$ that maps each q value to a D-dimensional vector, where D is the TTM model's hidden dimension. This prompt-consistency embedding is then added to the TTM condition vector, which further modulates all transformer blocks in the TTM model through AdaLN, providing scale-and-shift parameters for layer normalization. During inference, we simply use $q = 9$ (highest consistency) as the prompt consistency condition, since we assume that the user's input text prompt has no consistency issue.

4. EXPERIMENTS

4.1 Datasets

We use the MTG-Jamendo dataset [18] and the MusicCaps dataset [3] in the experiments. **MTG-Jamendo** is a large-scale music dataset collected from Jamendo music, which contains more than 55,000 audio tracks with genre, instrument, and mood/theme tags. **MusicCaps** contains 5,521 10-second music clips from the AudioSet dataset [33], each with a free text caption written by musicians. It has been widely used to evaluate recent TTM systems [9–11].

As a pre-processing step, we first cut all the audio in the MTG-Jamendo dataset into 10-second clips. Then, we adopt the official dataset split (`split-0`) by dividing it into a 251K training set and a 90K test set. For each clip, we apply LP-MusicCaps [17] with five different random seeds to generate five captions. Note that during this process, we did **not** make use of MTG-Jamendo's music tags, since we focus on a fully unsupervised TTM scenario (as mentioned in Section 3).

[1] https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[2] In other words, even in the training set, the number of samples with $q = 9$ does not necessarily equal to those with $q = 0$.

Dataset	MusicCaps						MTG-Jamendo test set					
Caption strategy	CLAP	PE-AV	CE	CU	PC	PQ	CLAP	PE-AV	CE	CU	PC	PQ
Baseline (w/o PromptCC)	0.1851	0.0467	5.913	6.747	4.983	6.544	**0.1986**	**0.1305**	6.124	6.950	5.103	6.755
PromptCC (Proposed)	**0.1975**	**0.0524**	**6.017**	**6.822**	4.758	**6.679**	0.1981	0.1283	**6.251**	**7.031**	4.974	**6.930**
PromptCC w/ BestCLAP	0.1950	0.0519	5.871	6.633	4.940	6.528	0.1920	0.1266	6.121	6.895	5.081	6.791
PromptCC w/o quantize	0.0571	-0.0378	5.772	6.505	**5.272**	6.417	0.0591	0.0073	5.736	6.467	**5.264**	6.375
CLAP conditioning	0.1619	0.0369	5.364	6.395	4.927	6.406	0.1514	0.1102	5.756	6.701	5.085	6.709
PQ conditioning	0.1848	0.0494	5.778	6.581	4.712	6.535	0.1910	0.1251	6.125	6.916	4.996	6.856
Consistency hard filtering	0.1663	0.0378	5.060	5.991	4.627	5.916	0.1861	0.1170	5.635	6.424	4.991	6.287

Table 1. Objective experiment results on MusicCaps (n=5,521) and MTG-Jamendo's test set (n=2,048). The best performance of each metric is highlighted in bold. For all the metrics, a higher number implies a better performance.

During training, we use MTG-Jamendo's training set for training, and use both MTG-Jamendo's test set and the entire MusicCaps dataset for evaluation. For MTG-Jamendo's test set, to speed up the evaluation, we use a subset of 2,048 samples randomly sampled from it for evaluation (the same subset is used across all experiments for consistency). For MusicCaps, we directly use its text captions for TTM.

4.2 Experiment Setup

The experiment setup and the hyperparameters mostly follow those of MeanAudio [8]. We consider the generation of 10-second music clips. During training, we train the MeanAudio model from scratch using the AdamW optimizer with a learning rate of 10^{-4}. Due to resource constraints, we set the batch size to 8 instead of 32. Model training is conducted with a two-stage training curriculum. In the first stage, the model is trained for 400K steps with the flow matching objective. Then, in the second stage, the model is fine-tuned for an additional 200K steps with the *mean flow matching* objective, which enables the one-step audio synthesis. In both stages, we randomly drop the condition with a dropout rate of 0.1 for the usage of CFG, with the guidance scale of 3.0. For the proposed prompt-consistency conditioning, we only add it to the TTM model in the second stage, which we empirically found to achieve a better performance.

4.3 Evaluation Metrics

We evaluate the proposed methods with objective and subjective metrics. The objective metrics are listed as follows:

Prompt-following. To evaluate TTM models' prompt-following ability, we adopt **CLAP similarity (CLAP)** and **PE-AV similarity (PE-AV)**. Both CLAP [20] and PE-AV [21] are multimodal encoders that can encode audio and text data. We employ these models to evaluate the semantic similarity between the text prompt and the generated audio.

Audio quality. To evaluate TTM models' generated audio quality, we adopt **Meta Audiobox Aesthetics (AES)** as an objective evaluator [19]. It is a Transformer model trained to predict four aspects of music: **Production Quality (PQ)**, **Production Complexity (PC)**, **Content Enjoyment (CE)**, and **Content Usefulness (CU)**. We use all four scores as objective metrics. Following MR-FlowDPO [27], we consider **PQ** to be the main metric, as it is particularly focused on the quality of the music.

In addition to objective metrics, we also conduct subjective test by recruiting human evaluators and asking their preferences in terms of both prompt-following and audio quality, as discussed in Section 4.6.

4.4 Baselines for Comparison

We compare the proposed prompt-consistency conditioning method (**PromptCC**) with several baselines and alternative methods, which are listed as follows:

1) **w/o PromptCC**: Train the MeanAudio model without the proposed PromptCC method. This is a direct ablation study that could reveal the effectiveness of the proposed method.

2) **PromptCC w/o quantize**: Use the proposed PromptCC method, but without quantizing the consistency score s. In particular, we directly prepend s to the text prompt. During inference, we set s to 1.0, i.e., assuming that the user prompt has no consistency issue.

3) **PromptCC w/ BestCLAP**: In this setting, we also use the proposed PromptCC; but during training, we always choose the caption that achieves the highest CLAP similarity score for training (instead of randomly selecting one from the five captions).

4) **CLAP conditioning**: Use the CLAP cosine similarity between audio and caption instead of PromptCC as the condition. This resembles the method of MR-FlowDPO [27].

5) **PQ conditioning**: Use Meta AES' PQ score instead of PromptCC as the condition. This also resembles the method of MR-FlowDPO [27].

6) **Consistency hard filtering**: A naive method that deals with the prompt consistency issue by removing audio with low prompt consistency (See Section 3.2). We set the consistency threshold to 0.8. This reduces the amount of training data from 251K to 117K (-53%).

4.5 Objective Experiment Results

The main experiment results are shown in Table 1. From the first two rows, the proposed PromptCC method outperforms the baseline (w/o PromptCC) in all metrics except

Metric	CMOS	t-value	p-value
Audio Quality	+0.467	$t(17)$=4.08	.0008
Prompt-Following	+0.289	$t(17)$=2.75	.014

Table 2. Human preference results at the participant level (N=18). CMOS > 0 indicates a preference toward the proposed method (PromptCC) over the baseline (w/o PromptCC). The proposed method is significantly preferred over the baseline in both audio quality and prompt-following metrics; both preferences survive Bonferroni correction over the two axes (α=.025).

PC in MusicCaps, confirming the effectiveness of providing the prompt consistency condition to the TTM model. In MTG-Jamendo's test set, PromptCC outperforms the baseline in CE, CU, and PQ, but underperforms the baseline in CLAP, PE-AV, and PC. These results indicate that, for in-domain text prompts, the effectiveness of the PromptCC method is mixed. Considering that the text prompts of MusicCaps are manually created, while the text prompts of MTG-Jamendo are auto-generated, we believe that the performance in MusicCaps is more representative in practical scenarios.

Then, we conduct two ablation studies. First, we compare PromptCC with "PromptCC w/ BestCLAP", which always uses the caption with the best CLAP score for training. Experiment results show a slight performance degradation in both audio quality and prompt-following ability. Second, we ablate the quantization step by comparing PromptCC with "PromptCC w/o quantize". Experiment results show a clear performance degradation in the CLAP and PE-AV score, suggesting that directly prepending the unquantized prompt-consistency score s to the text prompt harms the TTM model's prompt-following ability. It also confirms that injecting the prompt consistency information via the quantized learnable embedding is more effective than via the text encoder with natural language.

Then, we evaluate two alternatives to the proposed PromptCC method: CLAP conditioning and PQ conditioning. Both methods condition the TTM model through a similar mechanism (a trainable embedding), but with different conditions (CLAP score or PQ score). However, both alternatives underperform the proposed PromptCC method. Noticeably, both CLAP conditioning and PQ conditioning include one of the objective evaluation metrics in the TTM conditions; yet they still underperform the proposed PromptCC method in their respective metrics (CLAP conditioning: 0.1619 vs. 0.1975 in CLAP score; PQ conditioning: 6.535 vs. 6.679 in PQ score). These results again confirm the superiority of the proposed method, which, by providing the prompt consistency information to the TTM model, alleviates the caption consistency issue caused by the music captioning model in the unsupervised learning setting.

Lastly, the consistency hard filtering, a naive approach that directly removes audio with inconsistent prompts, significantly degrades the TTM performance in all metrics.

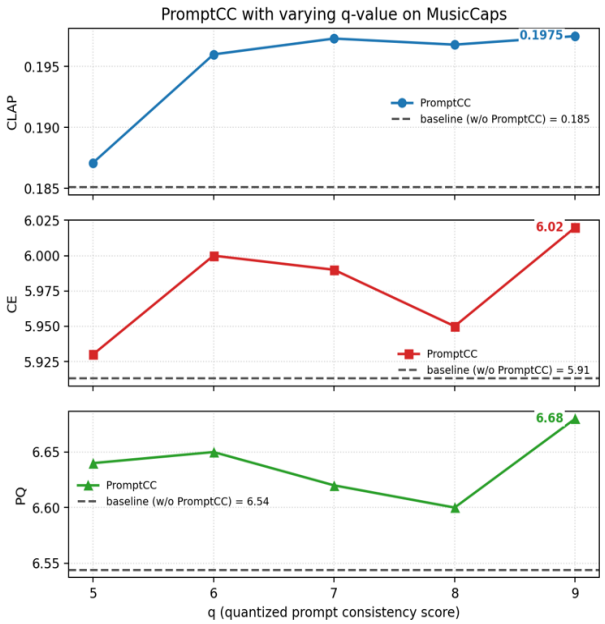

Figure 2. Experiment results of the proposed PromptCC method with varying q-values. Dotted lines (at the bottom of each subplot) denote the baseline (w/o PromptCC) model's performance. From top to bottom: CLAP score, AES CE score, and AES PQ score. For $q \geq 5$, varying q-value does not significantly change the generated audio's quality and prompt adherence.

Such results suggest that simply filtering out audio with inconsistent prompts is not the best strategy, possibly because it reduces the number of available training data pairs. This could result in a drop in TTM performance, as suggested in [19, 26]. The proposed PromptCC method, on the other hand, addresses the inconsistent prompts issue by providing the TTM model with the quantized consistency score, which implicitly allows the TTM model to handle such an issue without discarding any training sample, thereby achieving better performance.

4.6 Subjective Evaluation

To complement the objective tests, we conducted a paired preference test that compares the proposed method (PromptCC) with the baseline (w/o PromptCC). We first randomly selected 30 text prompts from the MusicCaps dataset [3], and used the two systems to perform TTM on these prompts under identical inference settings. Then, we recruited 18 participants to take the preference test. Participants were recruited via online posting, all with informed consent about the purpose of this test and that their responses are only used for research purposes. Participants listened via headphones in a quiet environment. For each question, we provided the participants with a text prompt and the corresponding audio clips generated by PromptCC and the baseline, respectively, but with a random order. The participant was then asked to rate their relative preference between the two audio clips in terms of **audio quality** and **prompt-following ability** using a seven-point CMOS

scale (integer ratings in $\{-3, \ldots, +3\}$), following the protocol of MR-FlowDPO [27]. Each participant rated a randomly chosen subset of 10 questions in total. As a result, we collected $18 \times 10 = 180$ ratings per metric in total.

Experiment results are shown in Table 2, where the proposed PromptCC is preferred over the baseline in both metrics, with CMOS scores of $+0.467$ (95% confidence interval: $[+0.225, +0.708]$, $p = 0.0008$) and $+0.289$ (95% confidence interval: $[+0.067, +0.511]$, $p = 0.014$), respectively. These preferences remain statistically significant after Bonferroni correction over the two axes (α=.025). We therefore observe a clear preference in audio quality and prompt-following ability for the proposed PromptCC method over the baseline, which confirms the effectiveness of the proposed PromptCC method in improving the TTM performance.

4.7 The Effect of q-value at Inference-Time

To further examine the effect of the q-value in the proposed PromptCC method, we again evaluate the performance of the proposed PromptCC method on the MusicCaps dataset, but with different q-values instead of fixing the q-value to 9. This allows us to examine the effect of the q-value conditioning during inference, potentially getting insights on why the proposed PromptCC method improves TTM performance in terms of prompt-following. Experiment results are shown in Figure 2. When the q-value is set to either $5, 6, 7, 8$, or 9, the TTM performance does not change significantly; all of them outperform the baseline (w/o PromptCC), which is visualized using horizontal dotted line. Note that we do not visualize the cases when $q < 5$ in the figure, as they achieve extremely poor TTM performance due to the lack of enough training data with these q-values. Since the distribution of consistency score s is highly skewed toward the right ($s \to 1.0$), there are very few data samples with $q < 5$ (only 313 samples). As a result, the TTM model does not have enough data to train the q embedding for $q < 5$, making the TTM performance drop significantly when $q < 5$.

From another perspective, when we focus on the cases of $q \geq 5$, using different q-values does not cause a lot of performance change during inference. In other words, the role of the q-value is not an indicator of the desired generated audio quality (unlike MR-FlowDPO [27]), but simply additional information that helps TTM model training. We suspect that the q-value actually **divides the training data into several categories (5, 6, 7, 8, 9) based on the potential extent of the prompt consistency issue**. During training, when the TTM model sees a prompt with $q = 5$, it understands that **it has to generate audio that not only fits the prompt, but also has the room to be interpreted differently**. On the other hand, when a prompt is paired with $q = 9$, the TTM model understands that it has to generate audio with less caption ambiguity. This does not mean that the audio generated with $q = 5$ should be of poorer quality (worse PQ) or more irrelevant to the text prompt (worse CLAP score). Instead, the q provides additional information to the TTM model that complements the text prompt, which, as we mentioned in Section 1, is only one of the possible ways to caption a particular audio clip.

5. LIMITATIONS AND FUTURE WORK

We acknowledge several limitations in this work. First, all experiments use a single flow-matching TTM backbone, so transfer to other TTM architectures remains unverified. Second, we only use one music captioning model (LP-MusicCaps [17]) in the experiments. In the future, our aim is to test the proposed PromptCC method with different TTM backbones and more music captioning models to further evaluate the effectiveness of the proposed method more comprehensively. We also plan to scale up the training data to examine the scalability of the proposed method.

6. CONCLUSION

In this work, we propose Prompt-Consistency Conditioning (PromptCC), a method that addresses the inconsistency issue of auto-generated text prompts for unsupervised TTM training, in which a music captioning model is used to generate captions automatically from unlabeled music clips. Recognizing the implicit problem that *there are multiple ways to caption a music clip*, the proposed PromptCC method first applies the same music captioning model multiple times to generate multiple captions. Then, we compute the average semantic similarity score s across all generated captions and quantize it into one of 10 q-levels. This q-value is then injected into the subsequent TTM model through a trainable embedding table, which allows the TTM model to understand the extent of the prompt consistency issue and to address it implicitly. Experiment results show that the proposed PromptCC achieves better TTM performance in terms of both audio quality (+0.467 CMOS) and prompt-following ability (+0.289 CMOS) than a baseline without PromptCC, along with several alternative methods: simply discarding audio clips with inconsistent prompts, providing the CLAP score between audio and the generated prompt, etc. These results confirm the effectiveness of PromptCC.

Furthermore, we also study the effect of the q-value during inference and show that changing the q-value between 5 and 9 does not significantly affect the generated audio quality and prompt adherence. These results provide insights into the actual role of the q-value: it is not a signal that controls the quality of the generated audio, but is an additional condition that supplements the text prompt. It provides information on caption consistency, which indicates how diversely a particular audio clip may be described. With such a condition, the TTM model can then be better trained, leading to improved TTM performance.

7. AI USAGE STATEMENT

We disclose the following uses of large language models (LLMs) during the preparation of this paper. First, we used Anthropic's Claude to assist with auxiliary scripts for data

organization and document handling (e.g., collating evaluation outputs). Second, we also used Claude to generate a draft of this paper. However, **all the contents of this paper have been carefully reviewed and corrected by the authors**, ensuring that there are no false references, factual inaccuracies, hallucinations, or unverified claims in the final manuscript. The authors assume full responsibility for the final content. In addition, all research ideas, method design, training and inference pipelines, experimental results, and analyses were developed independently by the authors.

8. REFERENCES

[1] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and controllable music generation," *Advances in Neural Information Processing Systems*, 2023.

[2] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," *arXiv preprint arXiv:2005.00341*, 2020.

[3] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi, M. Sharifi, N. Zeghidour, and C. Frank, "MusicLM: Generating music from text," *arXiv preprint arXiv:2301.11325*, 2023.

[4] L. Lin, G. Xia, J. Jiang, and Y. Zhang, "Content-based controls for music large language modeling," in *Proc. of the Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2024, pp. 783–790.

[5] R. Yuan *et al.*, "Yue: Scaling open foundation models for long-form music generation," *CoRR*, vol. abs/2503.08638, 2025. [Online]. Available: https://arxiv.org/abs/2503.08638

[6] Z. Evans, J. D. Parker, C. J. Carr, Z. Zukowski, J. Taylor, and J. Pons, "Stable audio open," in *Proc. of the IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.

[7] K. R. Prajwal, B. Shi, M. Lee, A. Vyas, A. Tjandra, M. Luthra, B. Guo, H. Wang, T. Afouras, D. Kant, and W.-N. Hsu, "MusicFlow: Cascaded flow matching for text guided music generation," in *Proc. of the Int. Conf. on Machine Learning (ICML)*, 2024, pp. 41 052–41 063.

[8] X. Li, J. Liu, Y. Liang, Z. Niu, W. Chen, and X. Chen, "MeanAudio: Fast and faithful text-to-audio generation with mean flows," *arXiv preprint arXiv:2508.06098*, 2025.

[9] S.-L. Wu, C. Donahue, S. Watanabe, and N. J. Bryan, "Music ControlNet: Multiple time-varying controls for music generation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2692–2703, 2024.

[10] Z. Novack, J. McAuley, T. Berg-Kirkpatrick, and N. J. Bryan, "DITTO: Diffusion inference-time T-optimization for music generation," in *Proc. of the Int. Conf. on Machine Learning (ICML)*, 2024.

[11] ——, "DITTO-2: Distilled diffusion inference-time T-optimization for music generation," in *Proc. of the Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2024.

[12] F. Schneider, O. Kamal, Z. Jin, and B. Schölkopf, "Moûsai: Efficient text-to-music diffusion models," in *Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 8050–8068.

[13] Z. Evans, J. D. Parker, C. J. Carr, Z. Zukowski, J. Taylor, and J. Pons, "Long-form music generation with latent diffusion," in *Proc. of the Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2024.

[14] Q. Huang, D. S. Park, T. Wang, T. I. Denk, A. Ly, N. Chen, Z. Zhang, Z. Zhang, J. Yu, C. Frank, J. Engel, Q. V. Le, W. Chan, Z. Chen, and W. Han, "Noise2Music: Text-conditioned music generation with diffusion models," *arXiv preprint arXiv:2302.03917*, 2023.

[15] S. Liu, A. S. Hussain, C. Sun, and Y. Shan, "Music understanding LLaMA: Advancing text-to-music generation with question answering and captioning," in *Proc. of the IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 286–290.

[16] Z. Kong *et al.*, "Improving text-to-audio models with synthetic captions," in *Synthetic Data's Transformative Role in Foundational Speech Models*, 2024.

[17] S. Doh, K. Choi, J. Lee, and J. Nam, "LP-MusicCaps: LLM-Based pseudo music captioning," in *Proc. of the Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2023.

[18] D. Bogdanov, M. Won, P. Tovstogan, A. Porter, and X. Serra, "The MTG-Jamendo dataset for automatic music tagging," in *Machine Learning for Music Discovery Workshop, International Conference on Machine Learning (ICML 2019)*, 2019.

[19] A. Tjandra, Y.-C. Wu, B. Guo, J. Hoffman, B. Ellis, A. Vyas, B. Shi, S. Chen, M. Le, N. Zacharov, C. Wood, A. Lee, and W.-N. Hsu, "Meta Audiobox Aesthetics: Unified automatic quality assessment for speech, music, and sound," *arXiv preprint arXiv:2502.05139*, 2025.

[20] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," in *Proc. of the IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[21] A. Vyas, H. Chang, C. Yang, P. Huang, L. Gao, J. Richter, S. Chen, M. Le, P. Dollár, C. Feichtenhofer, A. Lee, and W. Hsu, "Pushing the frontier of audiovisual perception with large-scale multimodal correspondence learning," *CoRR*, vol. abs/2512.19687, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2512.19687

[22] H. Chen, Y. Jiang, G. Ma, C. Hao, S. Wang, J. Yao, Z. Ning, M. Meng, J. Luan, and L. Xie, "DiffRhythm+: Controllable and flexible full-length song generation with preference optimization," *arXiv preprint arXiv:2507.12890*, 2025.

[23] Z. Ning, H. Chen, Y. Jiang, C. Hao, G. Ma, S. Wang, J. Yao, and L. Xie, "DiffRhythm: Blazingly fast and embarrassingly simple end-to-end full-length song generation with latent diffusion," *arXiv preprint arXiv:2503.01183*, 2025.

[24] X. Mei, C. Meng, H. Liu, Q. Kong, T. Ko, C. Zhao, M. D. Plumbley, Y. Zou, and W. Wang, "Wavcaps: A chatgpt-assisted weakly-labelled audio captioning dataset for audio-language multimodal research," *IEEE ACM Trans. Audio Speech Lang. Process.*, vol. 32, pp. 3339–3354, 2024. [Online]. Available: https://doi.org/10.1109/TASLP.2024.3419446

[25] X. Zhu, W. Tian, X. Wang, L. He, X. Wang, S. Zhao, and L. Xie, "Cosyaudio: Improving audio generation with confidence scores and synthetic captions," *CoRR*, vol. abs/2501.16761, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2501.16761

[26] C. Li *et al.*, "QA-MDT: Quality-aware masked diffusion transformer for enhanced music generation," in *Proc. of the Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 2025, pp. 10 135–10 143.

[27] A. Ziv, S. Chen, A. Tjandra, Y. Adi, W.-N. Hsu, and B. Shi, "MR-FlowDPO: Multi-reward direct preference optimization for flow-matching text-to-music generation," *arXiv preprint arXiv:2512.10264*, 2025.

[28] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.

[29] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, K. Lacey, A. Goodwin, Y. Marek, and R. Rombach, "Scaling rectified flow transformers for high-resolution image synthesis," in *Proc. of the Int. Conf. on Machine Learning (ICML)*, 2024.

[30] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, D. Valter, S. Narang, G. Mishra, A. Yu, V. Zhao, Y. Huang, A. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei, "Scaling instruction-finetuned language models," *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.

[31] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-Networks," in *Proc. of EMNLP*, 2019.

[32] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *CoRR*, vol. abs/2207.12598, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2207.12598

[33] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio set: An ontology and human-labeled dataset for audio events," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 776–780.